Supervyse OPF Intel Platform Porting Guide

Revision 1.5

07/2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1 Intel At Scale Debug (ASD) Porting Guide 7
1.1 Introduction 7
1.2 Configure the GPIO settings in the ASD 8
1.2.1 ASD Signal name in Intel document 8
1.2.2 The string (signal) and index utilized in the ASD source code 10
1.2.3 The gpio configuration in the device tree 12
1.2.4 JSON file for ASD 13
1.2.4.1 Configure the JSON file in the Entity-Manager recipe**Error! Bookmark not defined.**
1.3 Table: Summary of GPIO Pins Used by ASD 15

2 Configurable Force Firmware Update Jumper (FFUJ) Porting Guide 17
2.1 Introduction 17
2.2 Terminology 18
2.3 Configuration 18

3 Front Panel LED Indication of BMC State Porting Guide 20
3.1 Introduction 20
3.2 Specification 20
3.3 Configuration 20
3.3.1 Enable Front Panel LED Indicatior 21
3.3.2 Enable LED Pins in dts 21
3.3.3 Disable Functions that occupy LED Pins 22
3.3.4 Set LED GPIOs for user space .....**Error! Bookmark not defined.**
3.4 Flow Chart 24

4 Virtual Wire (vGPIO) Porting Guide 25
4.1 Introduction 25
4.2 Bootloader 25
4.3 Linux Kernel 25
4.4 User Space 26

5 Firmware Layout 29
5.1 Introduction 29
5.2 Flash with Single Image Layout 29
5.2.1 64MB 29
128MB 29
5.3 Flash size and partition offset 30
5.3.1 Parameter description: 32
5.3.2 auto-generated-confs.cfg 32

6 The Board ID Porting Guide 33
6.1 Introduction 33
6.2 SGPIO Configuration 33

7 OP-TEE & AST2600 TrustZone Porting Guide 34
7.1 How to enable OP-TEE 34
7.2 optee-os 35
7.3 Linux Kernel 35
7.4 Linux User Space 36

7.5 Verification 36
7.6 Reference 37

8 Power Control Porting Guide 38

8.1 Chassis power control Source code 38
8.2 Functionality of x86-power-control ..... **Error! Bookmark not defined.**
8.3 OEM configuration for x86-power-control 38
8.3.1 x86-power-control support/customization 38
8.3.1.1 GPIO configuration ..... **Error! Bookmark not defined.**
8.3.1.2 Warm reset configuration . **Error! Bookmark not defined.**
8.3.2 GPIO names and pin numbers concepts ..... **Error! Bookmark not defined.**
8.3.2.1 Build time configuration in dts **Error! Bookmark not defined.**
8.3.3 GPIO Passthrough ..... **Error! Bookmark not defined.**
8.3.3.1 Build time configuration in uboot/kernel **Error! Bookmark not defined.**
8.4 x86-power-control Verification 44

9 SOL/Obmc-console Porting Guide 46

9.1 Introduction 46
9.2 Architecture 46
9.3 Configuring obmc-console 47
9.3.1 For OPF G3 47
9.3.1.1 UART and UART Routing 48
9.3.1.2 UART configuration 48
9.3.2 For OPF G4 49
9.3.2.1 UART and UART Routing 49
9.3.2.2 UART configuration 50
9.3.2.3 Example: The host UART is ttyS0, and the UART routing is not required 51
9.4 Bring-up of the OBMC-Console SystemD Service 51
9.5 Verification 51
9.6 Default value of Parameters of IPMI SOL 52

10 Host Port 80h POST Code Direct to BMC GPIO/SGPIO Porting Guide 54

10.1 Introduction 54
10.2 Intel CRB 54

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2024-01-17	Edwin Liu	Edwin Liu	Initial version
1.1	2024-06-20	Wesley Wang	HungFu Lee	Add Firmware layout
1.2	2024-07-01	Youmin Chen	Edwin Liu	Add Configurable SGPIO
1.3	2024-07-26	Nick Lin	Austin Su	OP-TEE & AST2600 TrustZone
1.4	2024-12-23	Sam Tsai	Nick Lin	ASD Porting Guide
1.5	2025-01-10	Alice Chen	Edwin Liu	Power Control porting Guide

Terminology

Term	Description
BHS	Birch Stream
ASD	At Scale Debug
GNR	Granite Rapids

1 Intel At Scale Debug (ASD) Porting Guide

1.1 Introduction

Intel® At-Scale Debug (ASD) is a Debug Connectivity method that allows for the Remote PC Debug through BMC JTAG interface. The base ASD version, source origin, and corresponding recipe locations are documented here.

- ASD version: 1.5.1
- ASD source code: https://github.com/Intel-BMC/asd
- bbappend recipe path: *<SDK>/meta-insyde/meta-intel/recipes-core/at-scale-debug*

In addition to the JTAG-related pins, some GPIOs will also be used. This chapter will introduce two parts:

- Configure the GPIO settings in the ASD
    - The GPIO pins used by ASD are often referred to by different names in documentation, and software implementations. We will introduce the differences in Intel's description of ASD regarding the libgpiod name.
    - Configuring GPIO in ASD using JSON Files:
        - This guide will introduce the process of using JSON files to configure GPIO pins within the ASD system.
- Unified comparison of ASD GPIO pin names in documents, dtsi and code.
    - To clarify this, we will use BHS as an example and provide a table summarizing the pin names and their corresponding characteristics. This will help users better understand the descriptions in Intel's documentation.
    - The Description of Table: Summary of GPIO Pins Used by ASD section offers further information.

1.2 Configure the GPIO settings in the ASD

- By referring to the following two sections, you can understand the descriptions of ASD pins, as referenced in both the Intel documentation and the ASD source code.
    - ASD Signal name in Intel document
    - The string (signal) and index utilized in the ASD source code
- ASD uses libgpiod to access the GPIO. Make sure your GPIO settings (GPIO line names in the device tree) are correct.
    - "The gpio configuration in the device tree" section offers further information.
- The json file for entity-manager
    - The Entity-Manager's JSON file may override the libgpiod settings of the ASD program. You can check the JSON file if necessary.
    - The "JSON file for ASD" section provides more information.

1.2.1 ASD Signal name in Intel document

- Table 5-1 in Section 5.1 Birch Stream Hardware Connection of the Intel document 570214_Intel_At-Scale_Debug_BMC-JTAG_Design_Reference_Rev1p8.pdf.

Table 5-1. Birch Stream Signal Description

Signal Name	Connects To	BMC I/O Type	BMC Direction	Target Voltage Level	Voltage Translation	Description
BMC_PREQ_N	CPU_PREQ_N	Push-Pull	O	1.0V	Yes	Run Control Pin - Control Connects CPU PREQ_N pin. Figure 5-1.
BMC_PRDY_N	CPU_PRDY_N	Input	I	1.0V	Yes	Run Control Pin - Control Connects CPU PREY_N pin. Figure 5-1.
BMC_CPU_PWRGD	CPUPWRGOOD	Input	I	1.0V	Yes	Used for final power indication and for FBRK detection as the breaks move through the different early break states.[1]
FBRK	FBRK_N	Push-Pull	O	1.0V	Yes	CPU Early Break. Figure 5-2. See Note 1.
BMC_PRSNT_IN_N	POD_PRSNT_N	Input	I	3.3V		Detects when a Debugger has been plugged into the debug port on the systems. Figure 5-2.
BMC_PLTRST_IN_N	PLTRST	Input	I	3.3V		Platform Reset. Other methods of detecting PLTRST, such as eSPI reset, may be used to indicate that the CPUs have been reset.

CPU0_AUX_PWRGOOD	CPU0_AUX_PWRGOOD (Legacy AUX_PWRGD)	Input	I	1.8V	Yes	Used for early power indication and for FBRK Detection as the breaks move through the different early break states. See Note 1.
S0_PWROK	S0_PWROK	Input	I	1.0V	Yes	Used for FBRK Detection, as the breaks move through the different early break states. See Note 1.
BMC_ASD_EN_DET	ASD_EN_DET	Input	I	3.3V		Used to detect the position of the ASD enable jumper. Can be used as an indication that the ASD application may be started.
BMC_JTAG_SEL	JTAG Logic	Output	O	Implementation specific	Implementation Specific	Implementation dependent. Recommended for isolating 3.3V chain when BMC controls other JTAG* devices.

Two specific parts of this table require special attention during the porting process Connect to:

- Signal name:
  - Signal Names will correspond to the GPIO setting of ASD in BMC, but this name may be different from the program name.
  - The Signal Names from Table 5.1 have been added to the "DOC#570214 Signal Name" column of Table: Summary of GPIO Pins Used by ASD
- Connects to:
  - Names that are relevant to similar ones in the circuit schematic.
- Users can determine the relationship between settings and the hardware schematic through the "Signal name" (for BMC) and "Connects to" (for BHS schematic) fields.

1.2.2 The string (signal) and index utilized in the ASD source code

- This section introduces the default mechanism for controlling GPIO in ASD and explains the correspondence between its parameters and the JSON file override mechanism. The details of overriding will be discussed in more detail in the section "JSON file for ASD".

- In this context, *${AT_SCALE_DEBUG_ROOT}* refers to the directory where the ASD source code is located. Two methods for locating the ASD source code are provided below.
    - The source code of ASD can be found at *<SDK>/build/tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/at-scale-debug/git-r0/git/.*
    - Or you can check the source code by using command: *devtool modify at-scale-debug*, and the source code is located at *<SDK>/build/workspace/sources/at-scale-debug*.

- Control GPIO indexs are defined at *${AT_SCALE_DEBUG_ROOT}/target/target_handler.h*.

```
#define ALL_TARGET_CONTROL_GPIOS(FUNC)                                  \
    FUNC(BMC_TCK_MUX_SEL)                                               \
    FUNC(BMC_PREQ_N)                                                    \
    FUNC(BMC_PRDY_N)                                                    \
    FUNC(BMC_RSMRST_B)                                                  \
    FUNC(BMC_CPU_PWRGD)                                                 \
    FUNC(BMC_PLTRST_B)                                                  \
    FUNC(BMC_SYSPWROK)                                                  \
    FUNC(BMC_PWR_DEBUG_N)                                               \
    FUNC(BMC_DEBUG_EN_N)                                                \
    FUNC(BMC_XDP_PRST_IN)                                               \
    FUNC(POWER_BTN)                                                     \
    FUNC(RESET_BTN)                                                     \
    FUNC(BMC_PWRGD2)                                                    \
    FUNC(BMC_PWRGD3)
```

- Default strings were initialized for every index in *${AT_SCALE_DEBUG_ROOT}/target/target_handler.c*

```
    strcpy_s(state->gpios[BMC_TCK_MUX_SEL].name,
             sizeof(state->gpios[BMC_TCK_MUX_SEL].name), "TCK_MUX_SEL");
    state->gpios[BMC_TCK_MUX_SEL].direction = GPIO_DIRECTION_LOW;
    state->gpios[BMC_TCK_MUX_SEL].edge = GPIO_EDGE_NONE;

    strcpy_s(state->gpios[BMC_PREQ_N].name,
             sizeof(state->gpios[BMC_PREQ_N].name), "PREQ_N");
    state->gpios[BMC_PREQ_N].direction = GPIO_DIRECTION_HIGH;
    state->gpios[BMC_PREQ_N].edge = GPIO_EDGE_NONE;
    state->gpios[BMC_PREQ_N].active_low = true;

    strcpy_s(state->gpios[BMC_PRDY_N].name,
```

- To illustrate, consider the following example:
  - Code:
    - strcpy_s(state->gpios[BMC_PREQ_N].name,
      sizeof(state->gpios[BMC_PREQ_N].name), " PREQ_N");
    - copy the string “PREQ_N" to the index "BMC_PREQ_N".
  - String:
    - e.g. PREQ_N
    - The string 'PREQ_N' within the ASD should correspond to the device tree (dtsi) description for GPIO control.
  - Index:
    - e.g. BMC_PREQ_N
    - It is possible to use a JSON file to change the names corresponding to the dtsi or indicate that the pin is not in use.
    - The JSON file will dynamically update the corresponding string according to the indexed (e.g. BMC_PREQ_N).
    - The details of JSON file will be discussed in more detail in the section "JSON file for ASD".

1.2.3 The GPIO configuration in the device tree

- In BHS, the dtsi path is located at:
  - *<SDK>*/meta-insyde/meta-intel/meta-bhs/recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/aspeed-bmc-intel-bhs.dtsi
- Take PREQ_N (GPIOL4 in dts) as an example:
  - In schematic PREQ_N(GPIOL4) is connected to CPU_PREQ_N. For more info about CPU_PREQ_N, please refer to Table 5.1 above.
- The GPIOs in BMC which are connected to ASD device can be defined on the fly by editing aspeed-bmc-intel-bhs.dtsi.

```
&gpio0 {
    status = "okay";
    gpio-line-names =
    /*A0-A7*/   "","","","","SMB_CPU_PIROM_SCL","SMB_CPU_PIROM_SDA","SMB_IPMB_STBY_LVC3_SCL","SMB_IPMB_STBY_LVC3_SDA",
    /*B0-B7*/   "","","","FM_CPU1_ERR2_LVT3_N","RGMII_BMC_RMM4_LVC3_R_MDC","RGMII_BMC_RMM4_LVC3_R_MDIO","FM_BMC_BMCINIT_R","
    /*C0-C7*/   "FM_FORCE_BMC_UPDATE_N","RST_RGMII_PHYRST_N","FM_TPM_EN_PULSE","","","","FM_CPU1_ERR0_LVT3_N","FM_CPU1_ERR1_
    /*D0-D7*/   "CPU_ERR0","CPU_ERR1","CPU_ERR2","PRDY_N","FM_SPD_SWITCH_CTRL_N","","","",
    /*E0-E7*/   "FM_SKT1_FAULT_LED","FM_SKT0_FAULT_LED","CLK_50M_CKMNG_BMCB","ID_BUTTON","","","","",
    /*F0-F7*/   "","FM_DUAL_PARTITION_MODE_N","FM_STANDALONE_MODE_N","FM_4S_8S_N_MODE","FM_NODE_ID0","FM_NODE_ID1","","",
    /*G0-G7*/   "FM_SMB_BMC_NVME_LVC3_ALERT_N","RST_BMC_HSBP_MUX_N","FP_LED_STATUS_GREEN_N","FP_LED_STATUS_AMBER_N","PS_PWRO
_IN_R2_N",
    /*H0-H7*/   "SGPIO_BMC_CLK_R","SGPIO_BMC_LD_R","SGPIO_BMC_DOUT_R","SGPIO_BMC_DIN","PLTRST_N","CPU_CATERR","","",
    /*I0-I7*/   "","JTAG_ASD_TDI_R","JTAG_ASD_TCK_R","JTAG_ASD_TMS_R","JTAG_ASD_TDO","FM_BMC_PWRBTN_OUT_R_N","FM_BMC_PWR_BTN
    /*J0-J7*/   "","","","","","","","",
    /*K0-K7*/   "","","","","","","","",
    /*L0-L7*/   "","","","","PREQ_N","TCK_MUX_SEL","","",
    /*M0-M7*/   "","","","","","","SPA_SOUT","SPA_SIN",
    /*N0-N7*/   "","","","","","","SPB_SOUT","SPB_SIN",
    /*O0-O7*/   "","","","","","","NMI_BUTTON","SPEAKER_BMC_R",
    /*P0-P7*/   "RESET_BUTTON","RESET_OUT","POWER_BUTTON","POWER_OUT","","","","LED_BMC_HB_LED_N",
    /*Q0-Q7*/   "","","","","","","","",
```

1.2.4 JSON file for ASD

- To support different hardware configurations, ASD offers a flexible mechanism for modifying GPIO settings via a JSON file. This JSON file also provides a comprehensive overview of the modified GPIOs.

- Take Intel BHS CRB for example, the JSON file (**AVC-Baseboard.json)** can be found in the following path.

    - <SDK>/meta-insyde/meta-intel/meta-bhs/recipes-platform/configuration/entity-manager/insyde_overlay/configurations-insyde/**AVC-Baseboard.json.**

AVC-Baseboard.json:

```
{
  "BMC_DEBUG_EN_N": {
    "PinType": "PIN_NONE"
  },
  "BMC_PWR_DEBUG_N": {
    "PinType": "PIN_NONE"
  },
  "BMC_RSMRST_B": {
    "PinType": "PIN_NONE"
  },
  "BMC_SYSPWROK": {
    "PinName": "FM_BMC_CPU_FBRK_OUT_R_N"
  },
  "BMC_XDP_PRST_IN": {
    "PinName": "DBP_PRESENT_IN_R2_N"
  },
  "BMC_PWRGD2": {
    "PinName": "S0_PWROK"
  },
  ...
  "Name": "ASD",
  "Type": "ASD"
},
```

- **PinName** should be the same as **GPIO line names in kernel dts**(aspeed-bmc-intel-bhs.dtsi).

    - Example1: **BMC_PWRGD2**

        - Its PinName "S0_PWROK", represents the GPIO line name in kernel dts.

    - Due to the difference of cusomter hardware designs, the GPIOs used by ASD may vary. Therefore, the PinName (a.k.a GPIO Names) in JSON file are highly customized.

- When **PinType** is set as "PIN_NONE", it means this GPIO pin is not in-use.

    - Example2: **BMC_DEBUG_EN_N**

    - Its PinType is set as "PIN_NONE", which means it is not in-use on BHS.

    - According to the Intel document 570214_Intel_At-Scale_Debug_BMC-JTAG_Design_Reference_Rev1p8.pdf, the BMC_DEBUG_EN_N pin is only used in Purely & Whitley platforms.

- Through dbus, users can examine the ASD-configured overrides for GPIO settings on the BMC console.
    - busctl introspect xyz.openbmc_project.EntityManager /xyz/openbmc_project/inventory/system/board/${Intel_CRB}_Baseboard/ASD
- Example:

```
busctl introspect xyz.openbmc_project.EntityManager /xyz/openbmc_project/inventory/system/board/BNC_Baseboard/ASD
```

```
root@bmc-mac0007e9345b24:~# busctl introspect xyz.openbmc_project.EntityManager /xyz/openbmc_project/inventory/system/board/BNC_Baseboard/ASD
NAME                                                TYPE      SIGNATURE RESULT/VALUE                FLAGS
org.freedesktop.DBus.Introspectable                 interface -         -                           -
.Introspect                                         method    -         s                           -
org.freedesktop.DBus.Peer                           interface -         -                           -
.GetMachineId                                       method    -         s                           -
.Ping                                               method    -         -                           -
org.freedesktop.DBus.Properties                     interface -         -                           -
.Get                                                method    ss        v                           -
.GetAll                                             method    s         a{sv}                       -
.Set                                                method    ssv       -                           -
.PropertiesChanged                                  signal    sa{sv}as  -                           -
xyz.openbmc_project.Configuration.ASD               interface -         -                           -
.Name                                               property  s         "ASD"                       emits-change
.Type                                               property  s         "ASD"                       emits-change
xyz.openbmc_project.Configuration.ASD.BMC_DEBUG_EN_N interface -        -                           -
.PinType                                            property  s         "PIN_NONE"                  emits-change
xyz.openbmc_project.Configuration.ASD.BMC_PWRGD2    interface -         -                           -
.PinName                                            property  s         "S0_PWROK"                  emits-change
xyz.openbmc_project.Configuration.ASD.BMC_PWR_DEBUG_N interface -       -                           -
.PinName                                            property  s         "FM_BMC_CPU_FBRK_OUT_R_N"   emits-change
xyz.openbmc_project.Configuration.ASD.BMC_RSMRST_B  interface -         -                           -
.PinType                                            property  s         "PIN_NONE"                  emits-change
xyz.openbmc_project.Configuration.ASD.BMC_SYSPWROK  interface -         -                           -
.PinType                                            property  s         "PIN_NONE"                  emits-change
xyz.openbmc_project.Configuration.ASD.BMC_XDP_PRST_IN interface -       -                           -
.PinName                                            property  s         "DBP_PRESENT_IN_R2_N"       emits-change
xyz.openbmc_project.Configuration.ASD.BusConfig0    interface -         -                           -
.BusNum                                             property  t         0                           emits-change
.BusType                                            property  s         "BUS_CONFIG_I3C"            emits-change
xyz.openbmc_project.Configuration.ASD.BusConfig1    interface -         -                           -
.BusNum                                             property  t         1                           emits-change
.BusType                                            property  s         "BUS_CONFIG_I3C"            emits-change
xyz.openbmc_project.Configuration.ASD.BusConfig2    interface -         -                           -
.BusNum                                             property  t         2                           emits-change
.BusType                                            property  s         "BUS_CONFIG_I3C"            emits-change
xyz.openbmc_project.Configuration.ASD.BusConfig3    interface -         -                           -
.BusNum                                             property  t         3                           emits-change
.BusType                                            property  s         "BUS_CONFIG_I3C"            emits-change
root@bmc-mac0007e9345b24:~#
```

1.3 Table: Summary of GPIO Pins Used by ASD

- The GPIO pin names and configurations discussed in the previous chapters are summarized in this table.
    - DOC#570214 Signal Name:
        - These names are listed in Table 5.1 of Section 5.1, "Birch Stream Hardware Connection," in Intel's document 570214_Intel_At-Scale_Debug_BMC-JTAG_Design_Reference_Rev1p8.pdf.
        - The 'Signal name' (for BMC) and 'Connects to' (for BHS schematic) fields help users understand how settings relate to the physical hardware components.
    - ASD source code:
        - Index:
            - Within the ASD source code, there is an index specifically designed to store the string representations of GPIO names
            - The JSON file will use this index to replace the existing name string.
        - default string:
            - It is hardcoded within the ASD.
            - The system will fail to operate if the provided strings do not correspond to the gpio-names defined in the dtsi.
            - Users can customize these settings by providing values in a JSON file.
                - More detail refers to: Configure the JSON file in the Entity-Manager recipe
        - More detail refers to: The string (signal) and index utilized in the ASD source code
    - The AVC-Baseboard.json for BHS platform were used as examples in this table to override the gpio settings.
    - GPIO-line name in aspeed-bmc-intel-bhs.dtsi
- The GPIO settings must match the descriptions in the dtsi for the system to function correctly.

Table: Summary of GPIO Pins Used by ASD

***DOC#570214* Signal Name**	**ASD source code**		BHS (AVC-Baseboard.json)	**GPIO-line name in aspeed-bmc-intel-bhs.dtsi**
	Index	default string		
BMC_PREQ_N	**BMC_PREQ_N**	**PREQ_N**		PREQ_N
BMC_PRDY_N	**BMC_PRDY_N**	**PRDY_N**		PRDY_N
BMC_CPU_PWRGD	**BMC_CPU_PWRGD**	**SIO_POWER_GOOD**		SIO_POWER_GOOD
BMC_PWR_DEBUG_N	**BMC_PWR_DEBUG_N**	**PWR_DEBUG_N**	PIN_NONE [a]	
BMC_PRSNT_IN_N	**BMC_PLTRST_B**	**PLTRST_N**		PLTRST_N
BMC_PLTRST_IN_N	**BMC_XDP_PRST_IN**	**XDP_PRST_N**	DBP_PRESENT_IN_R2_N	DBP_PRESENT_IN_R2_N
S0_PWROK	**BMC_PWRGD2**	**BMC_PWRGD2**	S0_PWROK	S0_PWROK
CPU0_AUX_PWRGOOD	**BMC_PWRGD3**	**BMC_PWRGD3**	PWRGD_CPU0_AUXPWRGD	PWRGD_CPU0_AUXPWRGD [b]
BMC_ASD_EN_DET [c]				
BMC_JTAG_SEL	**BMC_TCK_MUX_SEL**	**TCK_MUX_SEL**		TCK_MUX_SEL
FBRK	**BMC_SYSPWROK**	**SYSPWROK [d]**	FM_BMC_CPU_FBRK_OUT_R_N	FM_BMC_CPU_FBRK_OUT_R_N
BMC_DEBUG_EN_N	**BMC_DEBUG_EN_N**	**DEBUG_EN_N**	PIN_NONE [a]	

[a] Purely & Whitley only

[b] BHS only

[c] Used to detect the position of the ASD enable jumper on BHS.

[d] In Purely call: SYSPWROK

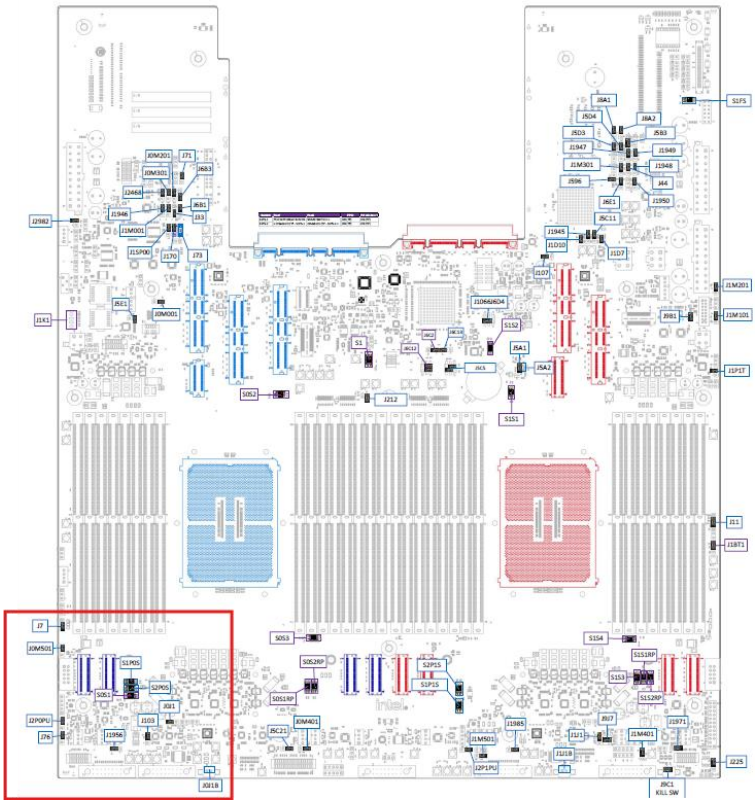
2Configurable Force Firmware Update Jumper (FFUJ) Porting Guide

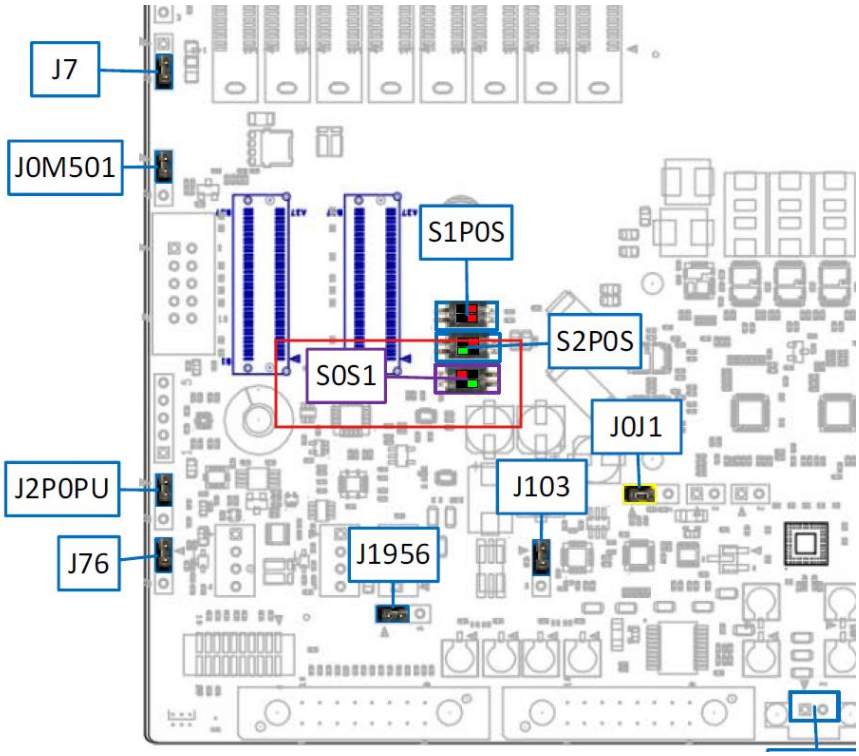
2.1 Introduction

When Force Firmware Update Jumper (FFUJ) is connected, BMC will trigger Safe-mode Boot. The U-boot will not boot into the Linux kernel, it will stop at CLI, waiting for the user to update firmware.

FFUJ pin on BHS is GPIOC6 and can be set by jumper S0S1-1. Please refer to Intel's Document: Beechnut City Theory of Operations (TOPs), Reference Number: 642714

STRAP	On DIE termination	Sw/ Jumper
SAFE_MODE_BOOT	PD	S0S1-1





2.2 Terminology

FFUJ	Force Firmware Update Jumper
CLI	Command-line Interface

2.3 Configuration

- Edit u-boot bbappend file

```
${SDK_root}/ meta-insyde/meta-intel/meta-bhs/recipes-bsp/u-boot/u-boot-aspeed-sdk_%.bbappend
```

- Config FFUJ

Disable FFUJ:

```
UBOOT_FFUJ_ENABLE = "0"
```

Enable FFUJ:

```
UBOOT_FFUJ_ENABLE = "1"
```

- Once GPIOC0 is pulling down before AC on, firmware will stop in Uboot in FFUJ mode

2.4 Logs

- If FFUJ is enabled and jumper is shorted, bmc will stop in U-boot as following logs.

```
U-Boot 2019.04 (Oct 23 2024 - 06:39:06 +0000)
```

```
SOC: AST2600-A3
RST: Power On
FMC 2nd Boot (ABR): Enable, Dual flashes, Source: Primary
eSPI Mode: SIO:Enable : SuperIO-2e
Eth: MAC0: RMII/NCSI, MAC1: RMII/NCSI, MAC2: RMII/NCSI, MAC3: RMII/NCSI
Model: Intel server board with AST2600 as the BMC
      Watchdog enabled
DRAM:  already initialized, Reserving 32768 Bytes for IRQ stack at: b95895e0
......
......

FFUJ: 1
ast#
```

3Front Panel LED Indication of BMC State Porting Guide

3.1 Introduction

During the BMC boot or BMC reset process, the System Status LED and System ID LED are used to indicate BMC boot process transitions and states. A BMC boot will occur when the AC power is first applied (DC power-on/off will not reset BMC). BMC reset will happen after the BMC firmware update, on receiving BMC cold reset command, and on BMC Watchdog initiated reset. The following table defines the LED states during the BMC Boot/Reset process.

3.2 Specification

#	State	Chassis ID	Status LED	Comment
1	Safe Mode (FFUJ)	Solid blue	Solid amber	Firmware update required to return to normal BMC operation.
2	Bootloader memory test failed	Blinking blue – 12 Hz	Solid amber	Non-recoverable condition. Contact your Intel's representative for information on replacing this motherboard.
3	Both firmware images are bad	Blinking blue – 6 Hz	Solid amber	Non-recoverable condition. Contact your Intel's representative for information on replacing this motherboard
4	BMC executing bootloader	Blinking blue – 3 Hz	Blinking green – 1 Hz	BMC is loading and authenticating the main firmware image. Server will be in this state for 4-8 seconds after BMC reset.
5	BMC Booting Linux*	Solid blue	Solid green	Solid green with solid blue after an AC cycle/BMC resets, indicates that the control has been passed from the bootloader to BMC Linux*. It will be in this state for around 4 seconds.
6	Normal operation	Off	Solid green	Indicates BMC Linux* has booted and manageability functionality is up and running. Fault/Status LEDs operate as per usual.
7	Failure recovery state	Solid blue	Blinking amber – 3Hz	BMC boots into u-boot after 3 times watchdog failures.
8	Manufacturing Test Mode	Blinking blue – 3 Hz	Blinking green – 3 Hz	BMC boots under manufacturing mode and can execute intrusive manufacturing commands.

3.3 Configuration

- Steps of configuration

1. Enable it in recipes
2. Enable LED pins in DTS
3. Disable functions that occupy LED pins
4. Set LED GPIOs for user space

- The default GPIO settings are compatible with Intel Birch Stream Platform
  Green: GPIOG2
  Amber: GPIOG3
  ID: GPIOB7

- Customers should adjust the relevant settings according to the actual hardware design.

3.3.1 Enable Front Panel LED Indicator

- Edit u-boot bbappend file Edit u-boot bbappend file
  ${SDK_root}/ meta-insyde/meta-intel/meta-bhs/recipes-bsp/u-boot/u-boot-aspeed-sdk_%.bbappend

- Enable:
  UBOOT_FP_LED_ENABLE = "1"

- Disable:
  UBOOT_FP_LED_ENABLE = "0"

3.3.2 Enable LED Pins in DTS

- Modify section "system-leds" in DTS
  Example:
  meta-insyde/meta-intel/meta-bhs/recipes-platform/u-boot-dts/u-boot-aspeed-sdk/insyde_overlay/arch/arm/dts/ast2600-intel.dts
  - Set label
  - Set LED pins by GPIO number (e.g. A0=0, A1=1, A7=7, B0=8, C0=16 etc.)
  - Add GPIO_ACTIVE_LOW if it is necessary

```
system-leds {
    compatible = "gpio-leds";
    green-led {
        label = "green";
        gpios = <&gpio0 50 GPIO_ACTIVE_LOW>;
        default-state = "blink";
    };
    amber-led {
        label = "amber";
        gpios = <&gpio0 51 GPIO_ACTIVE_LOW>;
        default-state = "off";
    };
    id-led {
        label = "id";
        gpios = <&gpio0 15 GPIO_ACTIVE_LOW>;
        default-state = "blink";
    };
};
```

3.3.3 Disable Functions That Occupy LED Pins

- Disable SD2 to release GPIOG2/G3 if it is tested on AST2600-evb

```
&sdhci_slot1 {
    //status = "okay";
    bus-width = <4>;
    pwr-gpios = <&gpio0 ASPEED_GPIO(V, 2) GPIO_ACTIVE_HIGH>;
    pwr-sw-gpios = <&gpio0 ASPEED_GPIO(V, 3) GPIO_ACTIVE_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_sd2_default>;
    sdhci-drive-type = <1>;
};
```

- Disable UART4 to release GPIOB7 if it is tested on AST2600-evb

```
&uart4 {
    //status = "okay";
};

&uart5 {
    status = "okay";
};
```

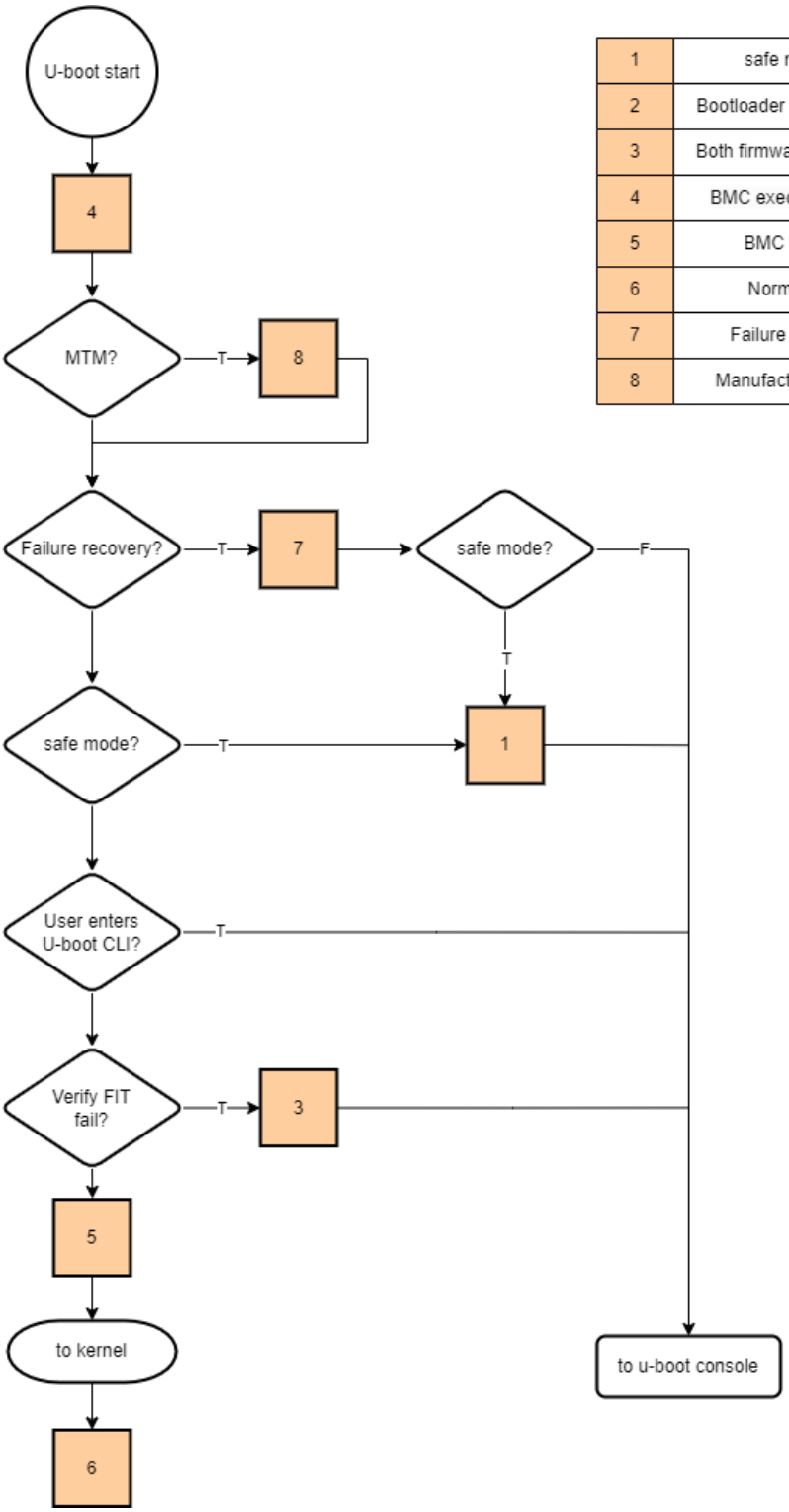
3.3.4 Set LED GPIOs for user space

Specification #6 “Normal Operation” is lit in user space. Please refer to Supervyse OPF Platform Porting Guide – LED Porting Guide

```
&uart4 {
    //status = "okay";
};

&uart5 {
    status = "okay";
};
```

3.4 Flow Chart



1	safe mode (FFUJ)
2	Bootloader memory test failed
3	Both firmware images are bad
4	BMC executing bootloader
5	BMC booting linux
6	Normal operation
7	Failure recovery state
8	Manufacturing Test Mode

4Virtual Wire (vGPIO) Porting Guide

4.1 Introduction

In this chapter, the porting of **Virtual Wire (vGPIO)** and a tool ***vwire_tool*** to verify are introduced.

Due to PCH is removed from Intel BHS CRB, so that **Virtual Wire (vGPIO)** can be used to handle the data transaction between BIOS and BMC. It is also known as channel 1 of eSPI protocol.

For more info about how to use **Virtual Wire (vGPIO)** please refer to the section **Intel Virtual Wire User Guide** in ***Supervyse OPF Intel Feature User Guide.*** For more info of what eSPI is and how data transaction handled between BMC and BIOS, please refer to the following Intel documents.

DOC#327432 Enhanced Serial Peripheral Interface (eSPI)
DOC#627433 eSPI Compatibility Specification – For Self-Boot Server Platforms

4.2 Bootloader

The driver of Virtual Wire (vGPIO) is not required in bootloader.

4.3 Linux Kernel

Kernel Config

Please apply the following Kernel configuration.

- CONFIG_GPIO_ASPEED_ESPI_VW=y

Kernel dts

Here are the examples to enable eSPI device in kernel dts, such as *aspeed-g6.dtsi* & *aspeed-bmc-intel-bhs.dtsi*.

aspeed-g6.dtsi

```
&espi {
    vwgpio: vwgpio@0 {
        compatible = "aspeed,ast2600-espi-vw-gpio";
        status = "disabled";
    };
};
```

aspeed-bmc-intel-bhs.dtsi

```
&espi {
    status = "okay";
```

```
};
&vwgpio {
    interrupts-extended = <&gic GIC_SPI 42 IRQ_TYPE_LEVEL_HIGH>;
    gpio-count = <32>;
    gpio-names-mask = <0x000001F1>;
    gpio-dir-mask = <0x00000FF0>;
    gpio-names = "VW_FM_BIOS_POST_CMPLT_N",
                 "VW_FM_DUAL_PARTITION_N", "VW_FM_STANDALONE_MODE_N",
                 "VW_FM_4S_8S_N_MODE", "VW_FM_NODE_ID_1", "VW_FM_NODE_ID_0";
    status = "okay";
};
```

Kernel dts Properties

- ***compatible***
  - aspeed,ast2600-espi-vw-gpio
  - This is the key word for the driver of virtual wire to probe.
- ***gpio-count***
  - Number of vGPIO lines. Unnamed vGPIO lines are included.
  - (min, max) = (1, 32). For the design of Virtual Wire (vGPIO) in AST2600 SoC, the maximum number of vGPIO supported is 32.
- ***gpio-names***
  - Similar to GPIO line names in GPIO driver. It is the name of each vGPIO line.
  - The array of human readable names to be used for each GPIO. Length can be less than or equal to ***gpio-count*** property. Refer to ***gpio-names-mask*** for usage.
- ***gpio-names-mask***
  - Bitmask to identify which vGPIO line maps to which index. 1'b means this vGPIO line is available. 0'b means this vGPIO line is not in use.
- ***gpio-dir-mask***
  - Bitmask to identify which vGPIO line is input and which is output. 1'b denotes output direction and 0'b means input direction.

4.4 User Space

vwire_tool is the tool designed by Insyde. It is useful for engineers to verify **Virtual Wire (vGPIO)**.

```
root@intel-obmc:~# vwire_tool
```

```
Copyright (c) 2013-2024, Insyde Software Corp. All Rights Reserved.

Usage :
   vwire_tool <option>
      --debug              : To show eSPI registers for debugging.
                           e.g. ESPI000, ESPI004, ESPI0A8, ESPI0C8.

      --sw-mode-on      : To enable sw mode of GPIO through VW Channel.
      --sw-mode-off     : To disable sw mode of GPIO through VW Channel.
      --set-dir-in <vGPIO>  : Set the direction of an eSPI vGPIO as input.
                           <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO-31].
      --set-dir-out <vGPIO> : Set the direction of an eSPI vGPIO as output.
                           <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO031].
      --set-dir-all <DIR>   : Set the direction for all eSPI vGPIO.
                           <DIR> : The hex value, e.g. 0x0000FFFF.
                                 - 1'b for eSPI output direction.
                                 - 0'b for eSPI input direction.

      --get-dir <vGPIO> : Get the direction of an eSPI vGPIO.
                        <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO-31].
      --get-dir-all     : Read the direction of all eSPI vGPIO.

      --set-val-low <vGPIO>  : Set the value of an eSPI vGPIO as value low.
                           <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO-31].
      --set-val-high <vGPIO> : Set the value of an eSPI vGPIO as value high.
                           <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO-31].
      --set-val-all <VAL>    : Set the value for all eSPI vGPIO.

                           <VAL>   : The hex value, 0x0000FFFF
                                 - 1'b for vGPIO value high
                                 - 0'b for vGPIO value low

      --get-val <vGPIO> : Get the value of an eSPI vGPIO.
                        <vGPIO> : [0 ~ 31] - [vGPIO-00 ~ vGPIO-31].
      --get-val-all     : Read the value of all vGPIO.
```

Here is the explanation of input parameters of ***vwire_tool***.

Input Parameters	**Description**
--debug	To show AST2600 eSPI registers for further debugging. e.g. ESPI000, ESPI004, ESPI0A8, ESPI0C8.
--sw-mode-on --sw-mode-off	To enable/disable sw mode of GPIO through VW Channel.
--set-dir-in <vGPIO> --set-dir-out <vGPIO>	Set the direction of an eSPI vGPIO as input / output. <vGPIO>:value [0 ~ 31] represents vGPIO0 ~ vGPIO31.

--set-dir-all <DIR>	Set the direction for all eSPI vGPIO. <VAL>: The 32-bit mapped hex value, each bit represents each vGPIO line. e.g. bit0 is vGPIO0, bit1 is vGPIO1, ...etc. 1'b is for eSPI output direction, and 0'b is for eSPI input direction.
--get-dir <vGPIO>	Get the direction of an eSPI vGPIO.<vGPIO>:value [0 ~ 31] represents vGPIO0 ~ vGPIO31.
--get-dir-all	Read the directions of all vGPIO lines.
--set-val-low <vGPIO> --set-val-high <vGPIO>	Set the value of an eSPI vGPIO as low/high. <vGPIO>:value [0 ~ 31] represents vGPIO0 ~ vGPIO31.
--set-val-all <VAL>	Set the value for all eSPI vGPIO. <VAL>: The 32-bit mapped hex value, each bit represents each vGPIO line. e.g. bit0 is vGPIO0, bit1 is vGPIO1, ...etc. 1'b is for vGPIO value high, and 0'b is for vGPIO value low.
--get-val <vGPIO>	Get the value of a vGPIO line. <vGPIO>:value [0 ~ 31] represents vGPIO0 ~ vGPIO31.
--get-val-all	Read the value of all vGPIO lines.

5Firmware Layout

5.1 Introduction

Modifying the firmware layout in OpenBMC requires a series of complex steps, including changes to the machine configuration file, the Linux DTS file, and U-Boot. To simplify these processes, Insyde has developed a mechanism called Simple Firmware Layout Configuration.

5.2 Flash with Single Image Layout

5.2.1 64MB

Region	Size	Offset	Content
spl	64 KB	00000000 ~ 00010000	
u-boot	960 KB	00010000 ~ 00100000	u-boot BootLoader
image-a	55808 KB	00100000 ~ 03780000	fitImage containing Kernel and rofs (squashfs-xz)
sofs	2048 KB	03780000 ~ 03980000	persistent NV filesystem, where immortal settings live.
rwfs	2048 KB	03980000 ~ 03FC0000	Other files created by programs (the ones not stored in SOFS)
certificate	64 KB	03FC0000 ~ 03FD0000	for HPM update
sdr	64 KB	03FD0000 ~ 03FE0000	Sensor Data Record
u-boot-env	64 KB	03FE0000 ~ 03FF0000	Parameters stored in u-boot environment
sdk_info	64 KB	03FF0000 ~ 04000000	Supervyse SDK information

128MB

Region	Size	Offset	Content
spl	64 KB	00000000 ~ 00010000	
u-boot	960 KB	00010000 ~ 00100000	u-boot BootLoader

image-a	55808 KB	00100000 ~ 03780000	fitImage containing Kernel and rofs (squashfs-xz)
sofs	2048 KB	03780000 ~ 03980000	persistent NV filesystem, where immortal settings live.
rwfs	2048 KB	03980000 ~ 03FC0000	Other files created by programs (the ones not stored in SOFS)
certificate	64 KB	03FC0000 ~ 03FD0000	for HPM update
sdr	64 KB	03FD0000 ~ 03FE0000	Sensor Data Record
u-boot-env	64 KB	03FE0000 ~ 03FF0000	Parameters stored in u-boot environment
sdk_info	64 KB	03FF0000 ~ 04000000	Supervyse SDK information

5.3 Flash size and partition offset

Flash size and partition offset configuration are located in the machine conf folder.
(1) Flash size setting

Take "**meta-insyde/meta-intel/meta-archercity /conf/machine/include/selected_flashsize.inc**" as an example:

It's flash size is set as 128MB:

FLASH_SIZE = "131072"

Currently two flash size are supported:

1. 64 MB: FLASH_SIZE = "65536"

2. 128 MB: FLASH_SIZE = "131072"

NOTE: The unit for those settings are "Byte".

(2) Partition offset configuration

For example, two possible partition offset configuration files are availabe for Intel platform:

meta-insyde/meta-intel/meta-archercity/conf/machine/include/flashlayout-64MB.inc

meta-insyde/meta-intel/meta-archercity/conf/machine/include/flashlayout-128MB.inc

The offset for 64MB is:

```
FLASH_SPL_OFFSET:flash-65536        = "0"
FLASH_SPL_OFFSET[name]              = "spl"

FLASH_UBOOT_OFFSET:flash-65536      = "64"
FLASH_UBOOT_OFFSET[name]            = "u-boot"

FLASH_KERNEL_ROFS_OFFSET:flash-65536 = "1024"
FLASH_KERNEL_ROFS_OFFSET[name]      = "image-a"
FLASH_KERNEL_ROFS_OFFSET[kentry]    = "1"

FLASH_SOFS_OFFSET:flash-65536       = "56832"
FLASH_SOFS_OFFSET[name]             = "sofs"

FLASH_RWFS_OFFSET:flash-65536       = "58880"
FLASH_RWFS_OFFSET[name]             = "rwfs"

FLASH_CERTIFICATE_OFFSET:flash-65536 = "65280"
FLASH_CERTIFICATE_OFFSET[name]      = "certificate"

FLASH_SDR_OFFSET:flash-65536        = "65344"
FLASH_SDR_OFFSET[name]              = "sdr"

FLASH_UBOOT_ENV_OFFSET:flash-65536   = "65408"
FLASH_UBOOT_ENV_OFFSET[name]        = "u-boot-env"
FLASH_UBOOT_ENV_OFFSET[ubootenv]    = "1"
FLASH_UBOOT_ENV_OFFSET[dual]        = "1"

FLASH_SDK_INFO_OFFSET:flash-65536    = "65472"
FLASH_SDK_INFO_OFFSET[name]         = "sdk-info"

FLASH_SEQ = "FLASH_SPL_OFFSET FLASH_UBOOT_OFFSET FLASH_KERNEL_ROFS_OFFSET
FLASH_SOFS_OFFSET FLASH_RWFS_OFFSET FLASH_CERTIFICATE_OFFSET FLASH_SDR_OFFSET
FLASH_UBOOT_ENV_OFFSET FLASH_SDK_INFO_OFFSET"
```

The offset for 128MB is:

```
FLASH_SPL_OFFSET:flash-131072        = "0"
FLASH_SPL_OFFSET[name]               = "spl"

FLASH_UBOOT_OFFSET:flash-131072      = "64"
FLASH_UBOOT_OFFSET[name]             = "u-boot"

FLASH_KERNEL_ROFS_OFFSET:flash-131072 = "1024"
FLASH_KERNEL_ROFS_OFFSET[name]       = "image-a"
FLASH_KERNEL_ROFS_OFFSET[kentry]     = "1"

FLASH_SOFS_OFFSET:flash-131072       = "56832"
FLASH_SOFS_OFFSET[name]              = "sofs"

FLASH_RWFS_OFFSET:flash-131072       = "58880"
FLASH_RWFS_OFFSET[name]              = "rwfs"

FLASH_CERTIFICATE_OFFSET:flash-131072 = "130816"
FLASH_CERTIFICATE_OFFSET[name]       = "certificate"

FLASH_SDR_OFFSET:flash-131072        = "130880"
FLASH_SDR_OFFSET[name]               = "sdr"

FLASH_UBOOT_ENV_OFFSET:flash-131072   = "130944"
FLASH_UBOOT_ENV_OFFSET[name]         = "u-boot-env"
FLASH_UBOOT_ENV_OFFSET[ubootenv]     = "1"
FLASH_UBOOT_ENV_OFFSET[dual]         = "1"
```

```
FLASH_SDK_INFO_OFFSET:flash-131072    = "131008"
FLASH_SDK_INFO_OFFSET[name]           = "sdk-info"

FLASH_SEQ = "FLASH_SPL_OFFSET FLASH_UBOOT_OFFSET FLASH_KERNEL_ROFS_OFFSET
FLASH_SOFS_OFFSET FLASH_RWFS_OFFSET FLASH_CERTIFICATE_OFFSET FLASH_SDR_OFFSET
FLASH_UBOOT_ENV_OFFSET FLASH_SDK_INFO_OFFSET"
```

NOTE: The unit for those settings are "Byte".

5.3.1 Parameter description:

FLASH_< partition _name>_OFFSET:flash-<Flash size>	Set the size of this partition.
FLASH_<Partion_name>_OFFSET[<Flag name>]	Yocto Variable Flag
FLASH_SEQ	The order of assembling the image.

5.3.2 auto-generated-confs.cfg

With the above setup, we will eventually generate by insyde-mtdparts.bbclass the confs.cfg file and write it to the U-Boot EVB for MTD partition configuration.

```
CONFIG_MTDIDS_DEFAULT="nor0=bmc"
CONFIG_MTDPARTS_DEFAULT="bmc:64k@0x0(spl),960k@0x10000(u-boot),55808k@0x100000(image-
a),2048k@0x3780000(sofs),6400k@0x3980000(rwfs),64k@0x3fc0000(certificate),64k@0x3fd000
0(sdr),64k@0x3fe0000(u-boot-
env),64k@0x3ff0000(sdk-info)"
CONFIG_BOOTARGS="console=ttyS4,115200n8 root=/dev/ram rw resetreason=0x13020
bonding.mode=1 bonding.fail_over_mac=0 bonding.miimon=100 bonding.downdelay=0
bonding.updelay=0 user_debug=0x08 fwproducti
d=0100 boardid=0100 bootflags=0 mtdparts=bmc:64k@0x0(spl),960k@0x10000(u-
boot),55808k@0x100000(image-
a),2048k@0x3780000(sofs),6400k@0x3980000(rwfs),64k@0x3fc0000(certificate),64k@0x3fd000
0(sdr),64k@0
x3fe0000(u-boot-env),64k@0x3ff0000(sdk-info)"
CONFIG_BOOTCOMMAND="bootm 20100000"
CONFIG_ENV_SIZE=0x8000
CONFIG_ENV_OFFSET=0x3FE0000
```

6The Board ID Porting Guide

6.1 Introduction

The Board ID of Intel CRB is programmed in FPGA and can be read from BMC SGPIO directly under u-boot.

SGPIO Pin Number	Direction	SGPIO Name
166	Input	FM_BOARD_SKU_ID0
168	Input	FM_BOARD_SKU_ID1
170	Input	FM_BOARD_SKU_ID2
172	Input	FM_BOARD_SKU_ID3
174	Input	FM_BOARD_SKU_ID4
176	Input	FM_BOARD_SKU_ID5

Here is the list of Board ID for each Intel CRB.

CRB Name	Board ID
Beechnut City	0x20
Beechnut City Modular	0x25
BNC XPV DUT	0x26
Avenue City	0x29
AVC XPV DUT	0x2A
AVC 2SPC DUT	0x2B
AVC 2SPC LRP	0x2D

6.2 SGPIO Configuration

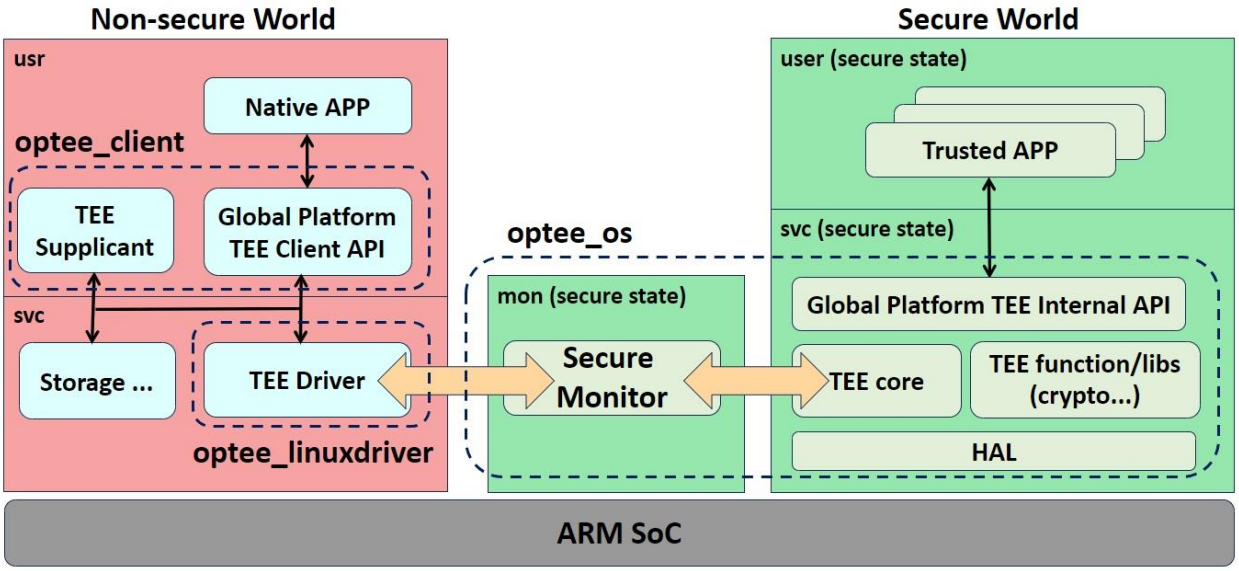
Please turn on SGPIO support in u-boot by setting **UBOOT_SGPIO_ENABLE** to "1" in ***u-boot-aspeed-sdk_%.bbappend***.

Enable SGPIO	UBOOT_SGPIO_ENABLE = "1"
Disable SGPIO	UBOOT_SGPIO_ENABLE = "0"

7OP-TEE & AST2600 TrustZone Porting Guide

OP-TEE is a Trusted Execution Environment (TEE) designed as companion to a non-secure Linux kernel running on Arm; Cortex-A cores using the TrustZone technology. OP-TEE implements TEE Internal Core API v1.3.1 which is the API exposed to Trusted Applications and the TEE Client API v1.0, which is the API describing how to communicate with a TEE. Those APIs are defined in the GlobalPlatform API specifications. For more information, please refer to OP-TEE Online Document.

OP-TEE & AST2600 TrustZone is only available for Intel BHS with **fw-security-features** enabled. The Linux kernel and OP-TEE OS are launched by u-boot automatically.



7.1 How to enable OP-TEE

In *local.conf.sample*, ***fw-security-features*** should be appended in **EXTRA_IMAGE_FEATURES** to have OP-TEE feature enabled.

```
EXTRA_IMAGE_FEATURES:append = " \
    fw-security-features \
    ras-offload-features \
    seamless-features \
    "
```

7.2 optee-os

- OP-TEE OS is a Trusted Execution Environment (TEE) designed as companion to a non-secure Linux kernel running on ARM.

- bb recipe: optee-os_git.bb

```
${OPF_ROOT}
└── openbmc-meta-intel
    └── meta-restricted
        └── recipes-optee
            └── optee-os
                ├── files
                │   ├── 0001-allow-setting-sysroot-for-libgcc-lookup.patch
                │   ├── 0002-Add-plat-aspeed-initial-commit.patch
                │   ├── 0003-port-plat-aspeed-from-3.9-to-3.12.patch
                │   ├── 0004-Revert-link.mk-add-missing-libgcc-to-ldargs.patch
                │   ├── 0005-Add-plat-aspeed-psci_system_reset.patch
                │   ├── 0006-core-ldelf-link-add-z-execstack.patch
                │   └── 0007-Enable-certificate-generation-in-OPTEE.patch
                └── optee-os_git.bb
```

- The rev. of optee-os is ***3d47a131bca1d9ed511bfd516aa5e70269e12c1d***, tag 3.12.0.

```
commit 3d47a131bca1d9ed511bfd516aa5e70269e12c1d (HEAD, tag: 3.12.0)
Author: Joakim Bech <joakim.bech@linaro.org>
Date:   Mon Dec 28 09:28:21 2020 +0100

    Update CHANGELOG for 3.12.0

    Update CHANGELOG for 3.12.0 and collect Tested-by tags.

    Signed-off-by: Joakim Bech <joakim.bech@linaro.org>
    Tested-by: Clement Faure <clement.faure@nxp.com> (mx6dlsabreauto)
    Tested-by: Clement Faure <clement.faure@nxp.com> (mx6dlsabresd)
    ...
    Tested-by: Joakim Bech <joakim.bech@linaro.org> (RPi3B-NFS)
    Tested-by: Ricardo Salveti <ricardo@foundries.io> (ZynqMP)
    Tested-by: Rouven Czerwinski <r.czerwinski@pengutronix.de> (imx-ccimx6ulsbcpro)
    Tested-by: Rouven Czerwinski <r.czerwinski@pengutronix.de> (imx-mx6qsabrelite)
    Tested-by: Sumit Garg <sumit.garg@linaro.org> (Developerbox)
    Tested-by: Victor Chong <victor.chong@linaro.org> (HiKey960 AOSP P)
    Tested-by: Volodymyr Babchuk <volodymyr_babchuk@epam.com> (Rcar H3)
    Tested-by: Volodymyr Babchuk <volodymyr_babchuk@epam.com> (Rcar H3/virtualization)
```

7.3 Linux Kernel

In order to have OP-TEE feature enabled, please enable the following Kernel Config.

```
CONFIG_TEE=y
CONFIG_OPTEE=y
```

7.4 Linux User Space

optee-client provides the TEE Client API as defined by the GlobalPlatform TEE standard. It is distributed under the BSD 2-clause open source license.

- **libteec.so**: It is the library that contains that API for communication with the Trusted OS.
- **tee-supplicant**: This is a daemon serving the Trusted OS in secure world with miscellaneous features, such as file system access.
- The rev. of optee-client, tag: 3.12.0

```
commit 347144b79964233e718347e3f0fbe89ab5be46ae (HEAD, tag: 3.12.0)
Author: Ruchika Gupta <ruchika.gupta@linaro.org>
Date:   Wed Jan 20 13:47:13 2021 +0530

    libckteec: serialize: fix check for ULONG type entries

    It is fine for callee to allocate more memory than what is
    needed for any attribute. Earlier for attributes of type
    ulong, a strict check was there which returned error
    if callee passed buffer greater than the required size.

    Signed-off-by: Ruchika Gupta <ruchika.gupta@linaro.org>
    Reviewed-by: Etienne Carriere <etienne.carriere@linaro.org>
```

optee-example are the demo applications of OP-TEE, such as data transaction between secure world(TEE) & unsecure world(REE). All OP-TEE example test applications are prefixed with *optee_example_*.

- The rev. of optee-example, tag : 3.12.0

```
commit 9a755dcf4d8ef6117af59dfd1b1a82315cee58ca (HEAD, tag: 3.12.0-rc1, tag: 3.12.0)
Author: Aleksandr Anisimov <a.anisimov@omprussia.ru>
Date:   Mon Dec 14 17:55:23 2020 +0300

    fix typo in comments for acipher, aes, hello_world and random examples

    Signed-off-by: Aleksandr Anisimov <a.anisimov@omprussia.ru>
    Reviewed-by: Joakim Bech <joakim.bech@linaro.org>
```

7.5 Verification

optee_example_hello_world

It is a very simple Trusted Application to answer a hello command and data transaction between non-secure world and secure world. Integer value 42 is bypassed from non-secure world to secure world and then plus 1. Finally at the end, send it back to non-secure world.

```
root@ast2600-default-tee:~# optee_example_hello_world
D/TA:  TA_CreateEntryPoint:39 has been called
D/TA:  __GP11_TA_OpenSessionEntryPoint:68 has been called
I/TA: Hello World!
Invoking TA to iD/TA:  inc_value:105 has been called
ncrement 42
I/TA: Got value: 42 from NW
I/TA: Increase value to: 43
TA incremented vI/TA: Goodbye!
alue to 43
D/TA:  TA_DestroyEntryPoint:50 has been called
```

```
root@ast2600-default-tee:~#
```

optee_example_random

To generate a random UUID using capabilities of TEE API, e.g. *TEE_GenerateRandom()*.

```
root@ast2600-default-tee:/usr/lib# optee_example_random
Invoking TA to gD/TA:  random_number_generate:74 has been called
enerate random UI/TA: Generating random data over 16 bytes.
UID...
TA generated UUID value = 0xff21812ce48a58f1f3b9ce9d799b9c1e
root@ast2600-default-tee:/usr/lib#
root@ast2600-default-tee:/usr/lib#
root@ast2600-default-tee:/usr/lib# optee_example_random
Invoking TA to gD/TA:  random_number_generate:74 has been called
enerate random UI/TA: Generating random data over 16 bytes.
UID...
TA generated UUID value = 0x41c9d9f3da8dd19b9234a7246836d2f
root@ast2600-default-tee:/usr/lib#
```

7.6 Reference

- ASPEED AST2600 A3 Datasheet Version 1.4 - Ch 10.2 AST2600 TrustZone
- OP-TEE Online Document - https://optee.readthedocs.io/en/latest/general/about.html
- OP-TEE Example - https://github.com/linaro-swg/optee_examples
- OP-TEE OS - https://github.com/OP-TEE/optee_os
- OP-TEE Client - https://github.com/OP-TEE/optee_client

8Power Control Porting Guide

8.1 Introduction

OPF power control implementation is based on Openbmc x86-power-control. It's implemented by Intel to support its x86 platforms. Insyde adds patches on it to support Insyde features.

Those functions provided by x86-power-control uses default JSON file (named: power-config-host0.json) for GPIO and time configurations.

8.2 Source code

The BB file location is at <SDK>/meta-phosphor/recipes-x86/chassis/x86-power-control_git.bb.

Insyde patch files location is at <SDK>/meta-insyde/meta-common/meta-obmc/recipes-x86/chassis/x86-power-control.

The x86-power-control configuration file is at <SDK>/meta-insyde/meta-intel/<Platform>/recipes-platform/x86-power-control-config/x86-power-control.

To extract x86-power-control source code execute:

```
devtool modify -w x86-power-control
```

8.3 x86-power-control GPIO configuration

x86-power-control through the GPIO pin to control the system power, so the GPIO setting is particularly important for x86-power-control, GPIO settings are divided into the following three parts.

- Configuration file
- Linux kernel and Linux kernel DTS
- u-boot

8.3.1 x86-power-control GPIO setting in configuration file

There is a configuration file to set the GPIO pins of x86-power-control.
When we use the command "devtool modify" to extract source code, the configuration file is in below path:

build/workspace/sources/x86-power-control/power-control-x86/config/power-config-host0.json.

It has two kinds of definitions, "gpio_configs" and "timing_configs", We are going to focus on "**gpio_configs**", gpio_configs defines all GPIO pins used by x86-power-control.

In gpio_configs, there are two types of gpio_configs. **GPIO** and **D-Bus**.

Below is an example for type of GPIO:

```
{
  "gpio_configs":[
```

```
  {
    "Name" : "IdButton",
    "LineName" : "ID_BUTTON",
    "Type" : "GPIO",
    "Polarity" : "ActiveLow"
  },
 ...
 ],
 ...
}
```

For the platform not having direct GPIO pin, we can use D-Bus based event monitor by using the type DBUS. Below is an example for type of D-Bus:

```
  {
    "Name": "PostComplete",
    "LineName": "POST_COMPLETE",
    "DbusName" : "xyz.openbmc_project.Settings",
    "Path" : "/xyz/openbmc_project/state/host0/post_complete",
    "Interface" : "xyz.openbmc_project.State.Host.PostComplete",
    "Property" : "PostCompleteInactive",
    "Type" : "DBUS"
  },
```

The following is a description of each item:

- Name

  A predefined keyword used by x86-power-control service to recognize a pin for special function. For this case, a gpio_configs with Name:"IdButton" means the pin is used as chassis id led button.

- LineName

  A keyword defined in kernel DTS file for the user space to operate the GPIO. For this case, "ID_BUTTON" must be defined in kernel DTS file for user space to use (The kernel DTS settings are described later).

- Type

  "GPIO" means it is a direct GPIO, “DBUS” is an indication that its source uses D-Bus.

- Polarity (Used in GPIO type)

  decide if a GPIO is active high or active low.

- DbusName (Used in D-bus type)

  D-Bus object of signal monitored.

- Path (Used in D-bus type)

  D-Bus path of signal monitored.

- Interface (Used in D-bus type)

  D-Bus interface of signal monitored.

- Property (Used in D-bus type)

D-Bus property of signal monitored.

Below is the table used in current power-config-host0.json, If the hardware and DTS design of the current platform is different from the configuration JSON file, appropriate modifications must be made.

Name	LineName	Usage	Note
"PowerOut"	"POWER_OUT"	Power button output to the host.	Mandatory.
"PowerOk"	"PS_PWROK"	System Power supply Power good signal detection of the host. Used to check power on/off.	Mandatory.
"ResetOut"	"RESET_OUT"	Reset button output to the host.	Mandatory.
"PostComplete"	"POST_COMPLETE"	The signal indicates the host BIOS POST is completed on the host. The function can be provided optionally by Insyde IPMI OEM command or GPIO pin. If USE_PLT_RST is not defined, deassertion of this pin(or OEM command) triggers warm reset event during power on. Implementation must ensure this pin(or OEM command) toggles during warm reset. If USE_PLT_RST is defined, see "Warm reset configuration" in the later section for details.	Optional. Remove this setting from power-config-host0.json if not used.
"NMIOut"	"NMI_OUT"	Front panel NMI button to the host.	Optional. Remove this setting from power-config-host0.json if not used.
"SioPowerGood"	"SIO_POWER_GOOD"	System CPU power good signal detection of the host. Similar to PS_PWROK, used to	Optional. Remove this setting from power-config-host0.json if not used.

		confirm if power on process is ok.	
"SioOnControl"	"SIO_ONCONTROL"	Currently x86-power-control does not use this pin to do any specific function.	Optional. Remove this setting from power-config-host0.json if not used.
"SIOS5"	"SIO_S5"	System S5 signal detection of the host. Assert/Deassert for S5/S0.	Optional. Remove this setting from power-config-host0.json if not used.
"PowerButton"	"POWER_BUTTON"	Front panel power button.	Optional. Remove this setting from power-config-host0.json if not used.
"ResetButton"	"RESET_BUTTON"	Front panel reset button.	Optional. Remove this setting from power-config-host0.json if not used.
"IdButton"	"ID_BUTTON"	This signal is the front panel ID LED button.	Optional. Remove this setting from power-config-host0.json if not used.
"NMIButton"	"NMI_BUTTON"	Front panel NMI button.	Optional. Remove this setting from power-config-host0.json if not used.

8.3.2 x86-power-control GPIO settings in Linux kernel and DTS

The **LineName** described in previous section is defined in kernel DTS. The configuration sets a binding between GPIO name and number. Changing the defined name requires modification on both kernel DTS and x86-power-control configuration JSON file.

- GPIO linename setup in DTS

The Linux kernel DTS file location is at <SDK>/build/workspace/sources/linux-aspeed/<platform>/arm/boot/dts/aspeed-bmc-<platform>-ast2600.dts

Below is an example for ID_BUTTON (GPIOE3):

```
…..
/*D0-D7*/      "CPU_ERR0","CPU_ERR1","CPU_ERR2","PRDY_N","FM_SPD_SWITCH_CTRL_N","","","",
/*E0-E7*/
      "FM_SKT1_FAULT_LED","FM_SKT0_FAULT_LED","CLK_50M_CKMNG_BMCB","ID_BUTTON","","","","",
/*F0-F7*/
      "","FM_DUAL_PARTITION_MODE_N","FM_STANDALONE_MODE_N","FM_4S_8S_N_MODE","FM_NODE_ID0",
"FM_NODE_ID1","","",
…..
```

Each GPIO pin used must have the linename set in the kernel DTS.

- GPIO passthrough

For Aspeed AST2600, there are 3 GPIO sets (6 GPIO pins) that support GPIO signal hardware passthrough.
They are:

GPIOP0(IN)->GPIOP1(OUT)

GPIOP2(IN)->GPIOP3(OUT)

GPIOP4(IN)->GPIOP5(OUT)

Basic register programming is required for them to work correctly in U-boot and kernel.

A passthrough GPIO set contains two pins: GPIO input and output. It is to pass the GPIO state from GPIO input pin directly to GPIO output pin when passthrough function enabled.

- GPIO passthrough in Linux kernel

kernel driver helps to enable/disable passthrough during libgpiod request/release.
Disable a passthrough set:

When the GPIO output pin is set as output high/low (E.g. requested by libgpiod object's request()), kernel disables the passthrough for that pin set and output the request value to the output pin.

Enable a passthrough set:

When the GPIO output pin is released (E.g. requested by libgpiod object's reset() or by freeing the object), the passthrough is enabled. This requires an OEM modification in kernel call back. It is described below.

Extract kernel code commands:

```
devtool modify -w linux-aspeed
```

The libgpiod's reset()/freeing object invokes the "aspeed_g6_gpio_disable_free()

" function in kernel. Currently, GPIOP1 and GPIOP3 passthrough have been enabled.

If other GPIO passthrough pins are required, change the code here.

File location is linux-aspeed/drivers/pinctrl/aspeed/pinctrl-aspeed-g6.c

```
#define GPIOP1 121
#define GPIOP3 123
static void aspeed_g6_gpio_disable_free(struct pinctrl_dev *pctldev,
                struct pinctrl_gpio_range *range,
                unsigned int offset)
{
    /*
     * If we're freeing GPIOP1 (121) or GPIOP3 (123) then re-enable the
     * pass-through mux setting; otherwise, do nothing.
     */
    if (offset != GPIOP1 && offset != GPIOP3)
        return;

    aspeed_gpio_disable_free(pctldev, range, offset);
}
```

- GPIO passthrough in u-boot

We want the system to have the passthrough function as soon as it is turned on, so we set up the passthrough function early in u-boot.

Extract U-boot code:

```
devtool modify -w u-boot-aspeed-sdk
```

Currently, the code enables GPIO passthrough of "GPIOP0 to GPIOP1", "GPIOP2 to GPIOP3" and "SIOPBI# to SIOPBO#".
If other GPIO passthrough pins are required, change the code here.

The file location is board/aspeed/ast2600_intel/intel.c, function is gpio_passthru_init().

```
static void gpio_passthru_init(void)
{
   writel(readl(SCU_BASE | SCU_4bc) |
         SCU_4bc_PASSTHRU0_MASK | SCU_4bc_PASSTHRU1_MASK,
         SCU_BASE | SCU_4bc);
   writel(readl(SCU_BASE | SCU_418) | SCU_418_PBIO_MASK,
         SCU_BASE | SCU_418);
}
```

8.3.3 X86-power-control timing setting

The timing_configs item in the configuration file sets the timings used by the x86-power-control.

```
...
  "timing_configs":{
     "PowerPulseMs": 200,
     "ForceOffPulseMs": 15000,
     "ResetPulseMs": 500,
     "PowerCycleMs": 5000,
     "SioPowerGoodWatchdogMs": 12000,
     "PsPowerOKWatchdogMs": 8000,
     "GracefulPowerOffS": 300,
     "WarmResetCheckMs": 500,
     "PowerOffSaveMs": 7000
    }
...
```

timing_configs properties table:

Name	Meaning
"powerPulseMs"	POWER_OUT pulse duration (ms) for power on/soft power off.
"forceOffPulseMs"	POWER_OUT pulse duration (ms) for force power off.

"resetPulseMs"	RESET_OUT pulse duration (ms) to reset the host.
"powerCycleMs"	Wait time (ms) between POWER_OUT off/on cycling action.
"sioPowerGoodWatchdogMs"	Timeout (ms) for the watchdog that wait for SIO_POWER_GOOD assert after PS_PWROK assert. Detection of power on failure. x86-power-control power down and log (journal) the event after the failure.
"psPowerOKWatchdogMs"	Timeout (ms) for the watchdog that waits for PS_PWROK assert after host power on is request. Detection of power on failure. x86-power-control power down and log (journal) the event after the failure.
"gracefulPowerOffS"	Timeout (s) for the watchdog that wait for PS_PWROK de-assert after host power graceful off is request. x86-power-control gives up the graceful power down when timeout occurs.
"WarmResetCheckMs"	Timeout (ms) to cancel warm reset after SIO_S5 is asserted. After the timeout, if SIO_S5 is not asserted, warm reset is confirmed. - Prevent false detection on the warm reset.
"PowerOffSaveMs"	Wait (ms) for PowerOffSaveTimeMs interval before saving chassis power state to storage.

8.4 x86-power-control basic Verification

We can use the D-bus provided by x86-power-control to verify the basic function of x86-power-control. The following D-bus commands are available for x86-power-contorl function verification.

- Power on

```
busctl set-
property xyz.openbmc_project.State.Host /xyz/openbmc_project/state/host0 xyz.openbmc_project.State.Host RequestedHostTr
ansition s "xyz.openbmc_project.State.Host.Transition.On"
```

Expected behavior: host power is powered on.

- Power off (hard)

```
busctl set-
property xyz.openbmc_project.State.Chassis /xyz/openbmc_project/state/chassis0 xyz.openbmc_project.State.Chassis Requeste
dPowerTransition s "xyz.openbmc_project.State.Chassis.Transition.Off"
```

Expected behavior: host power is powered off.

- Reset

```
busctl set-
property xyz.openbmc_project.State.Host /xyz/openbmc_project/state/host0 xyz.openbmc_project.State.Host RequestedHostTr
ansition s xyz.openbmc_project.State.Host.Transition.ForceWarmReboot
```

Expected Behavior: Reset host power.

- Power cycle (hard)

```
busctl set-
property xyz.openbmc_project.State.Chassis /xyz/openbmc_project/state/chassis0 xyz.openbmc_project.State.Chassis RequestedPowerTransition s "xyz.openbmc_project.State.Chassis.Transition.PowerCycle"
```

Expected Behavior: Power cycle the host.

- Graceful power off

```
busctl set-
property xyz.openbmc_project.State.Host /xyz/openbmc_project/state/host0 xyz.openbmc_project.State.Host RequestedHostTransition s "xyz.openbmc_project.State.Host.Transition.Off"
```

Expected behavior: host power is graceful power off.

- Graceful power cycle

```
busctl set-
property xyz.openbmc_project.State.Host /xyz/openbmc_project/state/host0 xyz.openbmc_project.State.Host RequestedHostTransition s "xyz.openbmc_project.State.Host.Transition.GracefulWarmReboot"
```

Expected Behavior: Host graceful power cycle.

- Power status

```
busctl get-
property xyz.openbmc_project.State.Host /xyz/openbmc_project/state/host0 xyz.openbmc_project.State.Host CurrentHostState
```

Expected Behavior: Return the current power status.

- Boot status

```
busctl get-
property xyz.openbmc_project.State.OperatingSystem /xyz/openbmc_project/state/os xyz.openbmc_project.State.OperatingSystem.Status OperatingSystemState
```

Expected Behavior: Return the current boot status.

- Turn on the x86-power-control log function in the console

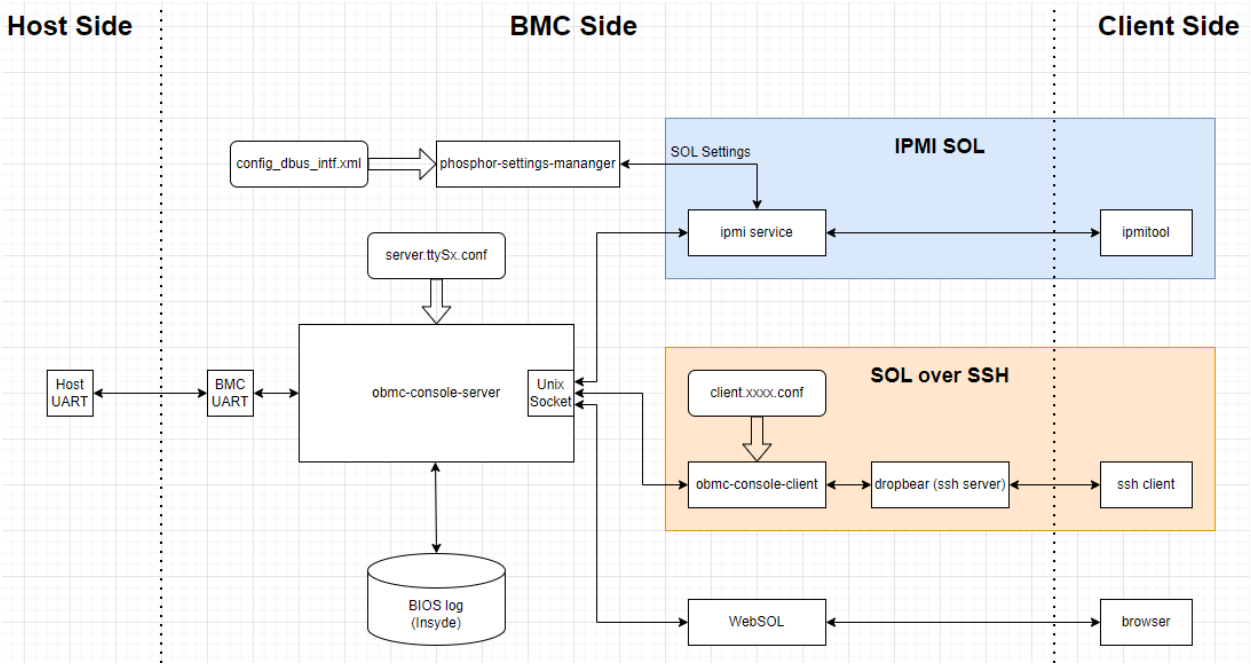
```
journalctl -p 7 -u xyz.openbmc_project.Chassis.Control.Power@0.service -f
```

9 SOL/Obmc-console Porting Guide

9.1 Introduction

On OpenBMC/OPF the SOL service is backed by obmc-console. The UART used by BMC to communicate with the console UART of the host can be altered by changing the configuration of obmc-console. This article is to illustrate how to change this configuration.

9.2 Architecture



Components in the Serial over LAN (SOL) subsystem:

- obmc-console-server: the service that opens the UART used for SOL functionality, and it also opens a Unix socket for other applications to communicate with the host. It is the core of the SOL service.
- phosphor-settings-manager: the service that keeps the settings of some software components in the BMC. The default value of these settings is stored in the config_dbus_intf.conf. Please refer to the Section 9.5 for more information
- IPMI service: The service to handle incoming IPMI requests, including getting/setting the parameter of the SOL, and activation/deactivation of the SOL.
- dropbear: the service to handle SSH connection.
- obmc-console-client: The service to handle the data flow between dropbear and the obmc-console.

- BIOS log: The feature implemented by Insyde. A timestamp is added at the beginning of every line marking the time that this line of code is received. Please refer to Section 1.4.3.5 BIOS log of the Supervyse OPF User Guide for more information.

The reconfiguration of SOL functionality consists of several parts:

1. Hardware UART used for SOL needs to be enabled. This requires modification to the device tree of the BMC.
2. Configuration of obmc-console.
3. Bring-up of obmc-console SystemD service.

9.3 Configuring obmc-console

To change the UART port that BMC used for SOL, two parts need to be changed. One part is the configuration file of the UART, and the way that the obmc-console service for that particular UART port is started. The UART needs to be enabled in the device tree as well. The configuration file of the obmc-console server is passed to the service as a parameter when the service is started.

The port that obmc-console will be monitoring can be passed to the service as a parameter as well, if it is not in the config file. In current implementation the host TTY information is passed as a command parameter.

The method of configuration of obmc-console is different for OPF G3 and OPF G4. This will be explained below.

9.3.1 For OPF G3

The configuration file of the obmc-console may contain the following fields:

Parameter	Function Description
baud	The baud rate of the UART that obmc-console is attached to.
upstream-tty	the UART that obmc-console server is attached to/listening for SOL.
socket-id	the name of the Unix socket opened by obmc-console server for communicating with other processes, for example, obmc-console-client, or dropbear/ssh. (Both obmc-console-server and obmc-console-client)
escape-sequence	the sequence to terminate/close SOL session. (obmc-console-client only)
logsize	Max size of the log file. (obmc-console.log)
logfile	Name of the log file. (obmc-console.log)
lpc-address	If the VUART is configured as the UART1 of the host, and is connected to host via LPC, this is the LPC address of the VUART.
sirq	If the VUART is configured as the UART1 of the host, and is connected to host via LPC, this is the serialized IRQ of the VUART.
local-tty	optional. This is the UART used for outputting the data from the host via SOL UART, and data inputted in this UART will be forwarded to the host via SOL UART.
local-tty-baud	The speed of the local-tty.

These parameters are for obmc-server-only unless specified.

9.3.1.1 UART and UART Routing

On Intel Eagle Stream platform UART routing is used. The Host uses UART1 (ttyS0) as its console output, and UART routing is used to redirect the traffic to UART3 (ttyS2). If the host UART needs to be changed, BIOS on the host, device tree, as well as the UART routing config needs to be modified.

The UART routing is now enabled and disabled directly by the unit file (obmc-console) of the SystemD. It is placed here: <SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-archercity/recipes-phosphor/console/obmc-console/obmc-console@.service

```
[Unit]
Description=Phosphor Console Muxer listening on device /dev/%I
BindsTo=dev-%i.device
After=dev-%i.device

[Service]
ExecStartPre=/usr/bin/sol-option-check.sh
ExecStartPost=/bin/sh -c 'echo -n "0x03450003" > /sys/bus/platform/drivers/aspeed-uart-routing/1e789098.uart-routing/hicra'
ExecStart=/usr/bin/env obmc-console-server --config {sysconfdir}/obmc-console.conf %i
ExecStopPost=/bin/sh -c 'echo -n "0" > /sys/bus/platform/drivers/aspeed-uart-routing/1e789098.uart-routing/hicra'
SyslogIdentifier=obmc-console-server
Restart=always

[Install]
WantedBy={SYSTEMD_DEFAULT_TARGET}
```

To change the UART routing, modify the parameter “0x03450003” in the line “ExecStartPost=/bin/sh -c ‘echo -n “0x03450003” > /sys/bus/platform/drivers/aspeed-uart-routing....”. The value “0x03450003” is to set UART routing between UART1 and UART3, and also between UART4 and IO1 (the default IO pin for UART1). For more detail, please refer to the register definition of the register HICRA in AST2600 datasheet.

Setting this parameter to 0 will disable the UART routing.

9.3.1.2 UART configuration

The UART configuration is stored in the configuration file "obmc-console.conf". It is located at the following folder:

<SDK_ROOT_DIR>/meta-openbmc-mods/meta-common/recipes-phosphor/console/

and its content:

```
baud = 921600
local-tty = ttyS3
local-tty-baud = 921600
```

The setting "baud = 921600" may be swapped by the shell script "/usr/bin/sol-option-check.sh" executed during the startup of the obmc-console.

In the OPF recipe used to build and configure obmc-console, the parameter OBMC_CONSOLE_HOST_TTY is used to assign which UART is used for SOL functionality. The recipe is:

<SDK_ROOT_DIR>/meta-openbmc-mods/meta-common/recipes-phosphor/console/obmc-console_%.bbappend

To change the UART used for SOL, add a new line to set OBMC_CONSOLE_HOST_TTY into the following recipe:

<SDK_ROOT_DIR>/meta-insyde/meta-intel/meta-archercity/recipes-phosphor/console/obmc-console_%.bbappend

For example, if the SOL is changed to ttyS1, then add this line:

```
OBMC_CONSOLE_HOST_TTY = "ttyS1"
```

to the aforementioned recipe.

9.3.2 For OPF G4

The configuration file of the obmc-console may contain the following fields:

Parameter	Function Description
baud	The baud rate of the UART that obmc-console is attached to.
upstream-tty	the UART that obmc-console server is attached to/listening for SOL.
console-id	the name of the Unix socket opened by obmc-console server for communicating with other processes, for example, obmc-console-client, or dropbear/ssh. It will also be inserted into the service name of obmc-console on Dbus. (Both obmc-console-server and obmc-console-client)
escape-sequence	the sequence to terminate/close SOL session. (obmc-console-client only)
lpc-address	If the VUART is configured as the UART1 of the host, and is connected to host via LPC, this is the LPC address of the VUART.
sirq	If the VUART is configured as the UART1 of the host, and is connected to host via LPC, this is the serialized IRQ of the VUART.
local-tty	optional. This is the UART used for outputting the data from the host via SOL UART, and data inputted in this UART will be forwarded to the host via SOL UART.
local-tty-baud	The speed of the local-tty.

These parameters are for obmc-server-only unless specified.

9.3.2.1 UART and UART Routing

On Intel Birch Stream platform UART routing is used. The Host uses UART1 (ttyS0) as its console output, and UART routing is used to redirect the traffic to UART3 (ttyS2). If the host UART needs to be changed, BIOS on the host, device tree, as well as the UART routing config needs to be modified.

The UART routing is now controlled by the obmc-console systemd service by invoking the shell script "sol-configure.sh" located at:

<SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console/

```
ROUTER=$(echo /sys/bus/platform/drivers/aspeed-uart-routing/*.uart-routing)
[ -L "$ROUTER" ] || exit 2

route() {
    echo -n "$1" > "$ROUTER/$2"
    echo -n "$2" > "$ROUTER/$1"
}

setup_routing() {
    echo "Enabling UART routing"

    route uart1 uart3
    route uart4 io1
```

```
}

setup() {
........
}

reset() {
........
}

teardown() {
   echo "Disabling UART routing"
   route uart1 io1
   route uart3 io3
   route uart4 io4
}
```

The setup_routing function configures the UART routing between different UARTs and between UART4 and the original IO pins of UART1, and teardown function disables it.
To change the UART routing, modify the parameter added when invoking “route” function.

Apart from controlling UART routing, this script also has the following function:

1. Setup function: Configure the baud rate of the upstream UART (host UART).
2. Reset function: Generate default server config.

9.3.2.2 UART configuration

The UART configuration is stored in the configuration file "server.ttyS2.conf". It is located at the following folder:

<SDK_ROOT_DIR>/openbmc-meta-intel/meta-common/recipes-phosphor/console/obmc-console/

and its content:

```
baud = 921600
local-tty = ttyS3
local-tty-baud = 921600
console-id = default
```

The setting "baud = 921600" may be swapped by the shell script sol-configure.sh located at:
<SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console/

It is executed during the startup of the obmc-console.

In the OPF recipe used to build and configure obmc-console, the parameter “OBMC_CONSOLE_HOST_TTY” is used to assign which UART is used for SOL functionality. The recipe is:

<SDK_ROOT_DIR>/openbmc-meta-intel/meta-common/recipes-phosphor/console/obmc_console_%.bbappend

To change the UART used for SOL:

1. add a new line to set OBMC_CONSOLE_HOST_TTY into the following recipe:
   <SDK_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console_%.bbappend

2. The server config file will need to be created (can be done by copying original config and renaming it) and included in the "SRC_URI" parameter of the obmc-console recipe.

3. The shell script file "sol-configure.sh" needs to be modified to change the name of the default config created to be the same as the one created in Step 2. The content of the server config file, and the setting of the UART routing in sol-configure.sh may also be changed if necessary.

9.3.2.3 Example: The host UART is ttyS0, and the UART routing is not required

1. add a new line to set OBMC_CONSOLE_HOST_TTY="ttyS0" into the following recipe:
   <SDK_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console_%.bbappend

2. Copy the file:
   <SDK_ROOT_DIR>/openbmc-meta-intel/meta-common/recipes-phosphor/console/obmc-console/server.ttyS2.conf
   to the following place:
   <SDK_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console/
   and rename it as server.ttyS0.conf
   Edit the recipe:
   <SDK_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console_%.bbappend
   by adding the following line:
   OBMC_CONSOLE_HOST_TTY="ttyS0"
   SRC_URI += "file://server.ttyS0.conf"
   SRC_URI:remove = "file://server.ttyS2.conf"

3. Edit the sol-configure.sh in the following directory:
   <SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes-phosphor/console/obmc-console/
   and replicate the content of the teardown() function into the function setup_routing()
   In the reset() function, there is a "ttyS2". Change it to ttyS0.

9.4 Bring-up of the OBMC-Console SystemD Service

On Intel BHS platform the bring-up of the obmc-console SystemD service is directly configured by the OPF recipe. It is placed in the following recipe:

<SDK_ROOT_DIR>/openbmc-meta-intel/meta-common/recipes-phosphor/console/obmc-console_%.bbappend

9.5 Verification

To verify the modified SOL is working, activate the SOL functionality:

1. IPMI SOL:
   ipmtool –H <bmc_ip> -I lanplus –U <user_id> -P <password> -C 17 sol activate
   Example:
   use the builtin username/password: root/0penBmc, and the BMC IP is 192.168.1.201
   ipmitool –H 192.168.1.201 -I lanplus –U root –P 0penBmc –C 17 sol activate

2. SOL over SSH:
   ssh –p2200 <user_id>@<bmc_ip>

Example:
user id is root, BMC IP is 192.168.1.201:
ssh –p2200 root@192.168.1.201

3. Web SOL:
   Go to the webpage of the BMC, log in, and then go to the left side of the screen. There's a list of labels. Click on the label "Operation", then a list of new labels should appear. Click on the label "SOL console" and then the web SOL will start.
   Please refer to Section 1.4.57 SOL console of the Supervyse OPF User Guide.

Then turn on the power of the host. The message output by BIOS during boot should start appearing on the screen. If the host allows user input (like, login or command prompt of linux, or UEFI shell) then characters keyed in through the keyboard should appear on the screen as well.

9.6 Default value of Parameters of IPMI SOL

There are some parameters that influence the operation of the IPMI SOL. These parameters are defined in the SOL Commands section of the IPMI 2.0 specification. The value of these parameters of IPMI SOL is stored in the phosphor-setting-manager. The object path of the dbus for each lan interface is in the format of "/xyz/openbmc_project/ipmi/sol/ethX", and the dbus interface is "xyz.openbmc_project.Ipmi.SOL". The default setting of the IPMI SOL is stored in the ipmidcfg configuration file located below:

For OPF G3:
<SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-archercity/recipes- platform/ipmidcfg/files/config_dbus_intf.xml

For OPF G4:
<SRC_ROOT_DIR>/meta-insyde/meta-intel/meta-bhs/recipes- platform/ipmidcfg/files/config_dbus_intf.xml

```
<special-tag content="/xyz/openbmc_project/ipmi/sol/eth0">
    <Interface>
        xyz.openbmc_project.Ipmi.SOL
    </Interface>
    <Properties>
        <Progress>
            <Type>
                uint8_t
            </Type>
            <Default>
                0
            </Default>
        </Progress>
        <Enable>
            <Default>
                'true'
            </Default>
        </Enable>
```

The mapping of the parameter:

Parameter in the Configuration File	Type of the parameter	Parameter in IPMITOOL
Enable	Boolean (value: true/false)	enabled
ForceEncryption	Boolean (value: true/false)	force-encryption

ForceAuthentication	Boolean (value: true/false)	force-authentication
Privilege	uint8_t	privilege-level
AccumulateIntervalMS	uint8_t	character-accumulate-level
Threshold	uint8_t	character-send-threshold
RetryCount	uint8_t	retry-count
RetryIntervalMS	uint8_t	retry-interval

The “Progress” Parameter in the configuration file is not used, so just ignore it.

10 Host Port 80h POST Code Direct to BMC GPIO/SGPIO Porting Guide

10.1 Introduction

For the purpose of hardware design, the Port **80h POST Code** from Host side should be redirected to 7-segment display on the motherboard via either **BMC's GPIO pins or BMC SGPIO pins**.

For Intel EagleStream and BirchStream CRB, Port **80h POST Code** are re-directed via SGPIOs.

BMC ---(SGPIO)--> FPGA --> 7-segment display

For SGPIOs in AST2600, only **SGPIO Bank A** can be used for redirection.

For GPIOs in AST2600, all GPIOs except GPIO bank T & U (input direction only) can be configured to receive signals redirected from the Port 80h POST Code.

Before verifying this feature, please make sure the GPIO/SGPIO driver are probed correctly or not. For more info, please refer to **GPIO Porting Guide** in **Supervyse OPF Platform Porting Guide**.

10.2 Intel CRB

[G3] Intel EagleStream CRB

Please edit the following configurations in ***u-boot-aspeed-sdk_%.bbappend***.

${OPF_ROOT}/meta-insyde/meta-intel/meta-archercity/recipes-bsp/u-boot/u-boot-aspeed-sdk_%.bbappend

To enable Port 80h POST Code redirect to SGPIO.

PORT80_ENABLE = "1"
PORT80_SGPIO_GPIO_SWITCH = "0"
PORT80_GPIO_INV_POL = "0"
PORT80_GPIO_BANK = "0"

To disable Port 80h POST Code redirect to both GPIO & SGPIO.

PORT80_ENABLE = "0"

Option	Description
PORT80_ENABLE	To enable/disable the re-direction of both GPIO & SGPIO. 0 for disable 1 for enable
PORT80_SGPIO_GPIO_SWITCH	Re-direct to GPIO or SGPIO? 0 for SGPIO 1 for GPIO

PORT80_GPIO_INV_POL	GPIO Polarity. 0 for active-high 1 for active-low
PORT80_GPIO_BANK	The GPIO bank where Port 80h POST Code is re-directed to. 0x0 for GPIO bank A 0x1 for GPIO bank B ... 0x19 for GPIO bank Z

[G4] Intel BirchStream CRB

Please enable the following configurations in u-boot's config. e.g. ***port80.cfg***.

${OPF_ROOT}/meta-insyde/meta-intel/meta-bhs/recipes-bsp/u-boot/files/port80.cfg

Here is an example to enable Port 80h POST Code redirect to **SGPIO**.

CONFIG_PORT80_ENABLE=y

CONFIG_PORT80_SGPIO=y

Here is an example to disable the redirection of Port 80h POST Code.

CONFIG_PORT80_ENABLE=n

Option	Description
CONFIG_PORT80_ENABLE	To enable and disable the redirection. y for enable n for disable
CONFIG_PORT80_SGPIO	Redirect to SGPIO? y for enable n for disable
CONFIG_PORT80_GPIO	Redirect to GPIO? y for enable n for disable
CONFIG_PORT80_GPIO_BANK	The GPIO bank where 80h POST Code is re-directed to. 0x0 for GPIO bank A 0x1 for GPIO bank B ... 0x19 for GPIO bank Z
CONFIG_PORT80_GPIO_INV_POL	GPIO Polarity y for active-low n for active-high